BIRLASOFT · FORGE FDE ACADEMY · SPRINT 1 · DAY 8 OF 15

Cosmos DB

Case state, and the one decision you cannot take back

Your three query patterns want three different partition keys. You get one, and it is immutable. Today is where that gets decided deliberately.

2 hours | Lab runs offline | P4 leads | Container live today

TODAY

Two hours

0:00 – 0:12	**Three queries, one key**	Pick the partition key. You cannot change it.
0:12 – 0:45	**The method**	RU · partitioning · document shape · change feed
0:45 – 1:30	**The lab**	Four experiments, four findings
1:30 – 1:50	**The defence**	Your partition key, defended against the other two
1:50 – 2:00	**Sprint contribution**	Container designed, change feed enabled

WHY TODAY MATTERS MORE THAN IT LOOKS Almost everything in this programme can be changed later. The partition key cannot — it is fixed when the container is created, and altering it means migrating the data. This is the one decision today that you either get right or live with.

START HERE · 4 MINUTES

Your brief lists three query patterns.

`get by case_id`	40,000 / month	*the hot path*
`list by applicant_id`	3,000 / month	*a handler checking history*
`filter by status`	6,000 / month	*the operations queue*

You get one partition key. It is fixed when the container is created.

In pairs. Which one do you pick — and what happens to the other two?

THE ANSWER

Nothing wins all three — and that is what a partition key is

Partition key	get by case_id	list by applicant	filter by status	Largest partition
case_id	1.5 RU	1,842 RU	1,842 RU	13%
applicant_id	1,842 RU	4.3 RU	1,842 RU	14%
status	1,842 RU	1,842 RU	1,032 RU	74% HOT

A PARTITION KEY MAKES ONE PATTERN A POINT READ AND EVERY OTHER ONE A SCAN That is not a limitation to work around. It is the mechanism. Cosmos is fast and cheap precisely because it knows which partition to go to — and when it does not know, it visits all of them.

The line that explains every number in that table

A cross-partition query costs RU per document EXAMINED, not per document returned. Asking for the six pending cases can read all four thousand — and you are charged for four thousand.

PART 1

RU is the currency — not milliseconds

~1 RU

Point read

`id + partition key`

The cheapest thing Cosmos does. Requires the caller to KNOW the partition key value.

~3–10 RU

Single-partition query

`filter includes the key`

Cheap. Scans one logical partition, which is usually a handful of documents.

~1 RU / doc

Cross-partition query

`key not in the filter`

Costs per document scanned across every partition. This is the one that ends up on an invoice.

ROUGHLY AN ORDER OF MAGNITUDE APART EACH TIME In Cosmos you provision RU, you consume RU, and the bill is RU. Latency follows from RU rather than the other way round. If you take one number from today, take the ratio between a point read and a cross-partition scan.

And writes: roughly 5x a read of the same size — which makes DOCUMENT SIZE a write-cost decision. We come back to that.

PART 1

So what do you actually do?

Partition on case_id. Serve the other two patterns with a second copy of the data.

Why case_id

High cardinality — 4,000 distinct values. Even distribution. And it makes the dominant pattern, 40,000 reads a month, a 1 RU point read.

list by applicant_id

A small materialised view keyed on applicant_id, maintained by the change feed. A point read instead of a 4,000-document scan.

filter by status

The same, or a view holding only the LIVE cases — a few dozen documents rather than four thousand, because most cases are complete.

The change feed keeps them current

This is what the change feed is for on day one, before it is your audit trail in Sprint 2.

THE SENTENCE TO REMEMBER You do not solve a second access pattern with a second index. You solve it with a second copy of the data, shaped for that question.

PART 1 · SCENARIO

Talk about this

DISCUSS · 7 MIN

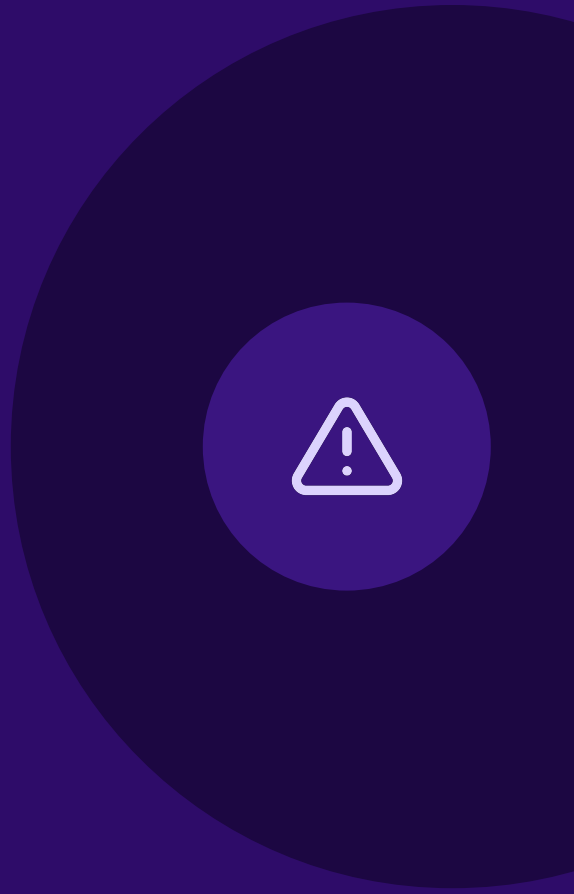
Six months in, the business asks for a new view: all cases for one employer, across all applicants. Nobody anticipated it. What are your options?

1. Query it directly. It is cross-partition and scans everything. What does that cost at 4,000 cases, and at 40,000?
2. Add another materialised view. Who maintains it, what does it cost to keep current, and what happens when the two copies disagree?
3. Change the partition key. What does that actually involve — and is there a version of this project where that is acceptable?

PART 2

The hot partition

429s while the dashboard shows headroom.



PART 2

The most confusing symptom in Cosmos

Morning peak. Forty handlers open the queue view and it auto-refreshes. 500 ops/sec × 8 RU = 4,000 RU/s demand. You provisioned 6,000 RU/s.

Partition key	Largest partition	Its demand	Ceiling	Result
case_id	13%	521 RU/s	1,000	fine
applicant_id	14%	549 RU/s	1,000	fine
status	74%	2,959 RU/s	1,000	429 THROTTLED

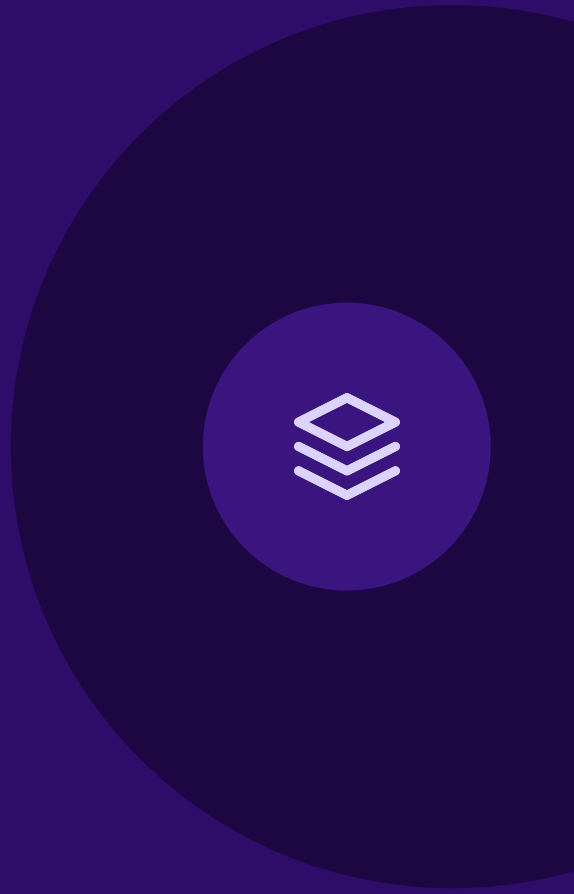
A SINGLE PHYSICAL PARTITION CANNOT EXCEED ROUGHLY 1,000 RU/S No matter what you provision at the container level. Partition by status and 74% of documents land on one partition, because most cases end up complete. You get 429s while the dashboard shows spare capacity — a partition key problem wearing a capacity costume.

The test for a good partition key: HIGH CARDINALITY and EVEN DISTRIBUTION. case_id has 4,000 values, evenly spread. status has 5, lopsided.

PART 3

Document shape

Cosmos charges write RU by size. Your status changes four times.



PART 3

One fat document, or hot and cold split

One fat document

- status, extracted_fields and ai_response together
- 3.0 KB
- 15 RU per write
- 239.5k RU a month on writes alone

Hot + cold split

- A small document with the status and a result_ref
- 0.5 KB
- 2.5 RU per write
- 40.0k RU a month — six times less

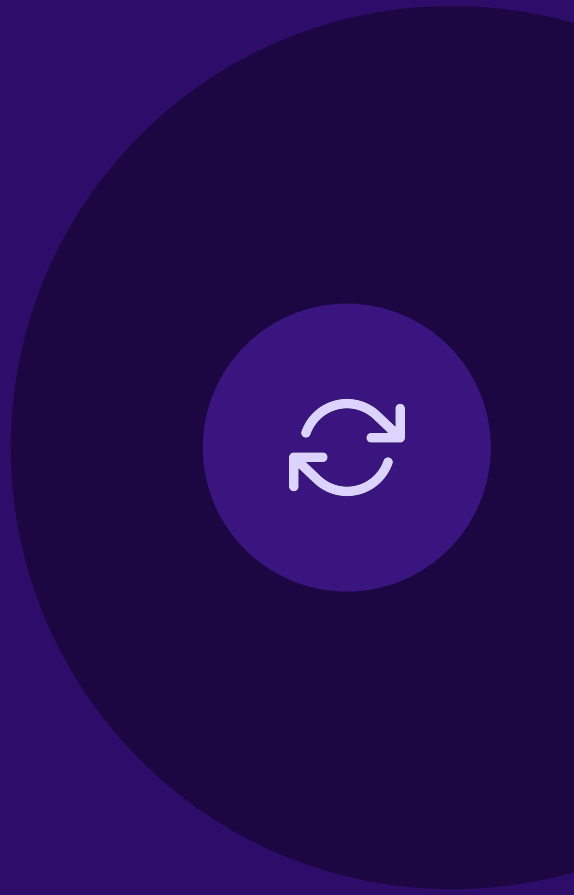
WHY THE FAT DOCUMENT IS EXPENSIVE A case changes status four times. With everything in one document, every status change rewrites the AI narrative and its citations — which have not changed and never will. You are paying to write the same 2.5 KB four times per case.

The cost of splitting: one extra read when somebody wants the full result. That is a point read — the cheapest thing Cosmos does. Separate what changes from what does not.

PART 4

The change feed

Enable it today. Understand it before Sprint 2.



PART 4

What your audit trail will — and will not — see

Change feed mode	Transitions	Observed	LOST
latest_version (the default)	200	67	133 (66%)
all_versions_and_deletes	200	200	0

THE DEFAULT GIVES YOU THE LATEST VERSION OF A CHANGED DOCUMENT, NOT EVERY VERSION A case moving pending → extracting → generating → complete faster than the polling interval is observed ONCE, as complete. Three intermediate states happened and were never captured.

For a status dashboard

This is fine. Nobody needs to see every intermediate state of a case that finished thirty seconds ago.

For your Sprint 0 audit trail

It is a hole. You cannot evidence a state you never recorded, and "the system moved through generating" is exactly what an investigation asks about.

SAY THIS TODAY Sprint 2 is where this gets discovered if nobody raises it now — and by then the audit trail is a commitment, not a design option.

PART 4

Two ways out, and they cost different things

All-versions mode

- Every version reaches the consumer
- No application change needed
- More feed volume, more cost
- Availability varies — check for your account and API

Write an explicit transition record

- An append-only event on every state change
- Never mutated, so nothing can be lost
- You control exactly what is captured
- One extra small write per transition

THE SECOND IS WHAT AN AUDITOR ACTUALLY WANTS Because it is a LOG rather than a stream of snapshots. A change feed tells you what a document looks like now; an append-only transition record tells you what happened and in what order. Those are different questions, and only one of them survives an investigation.

This also connects to Sprint 0 Day 6: the audit record is written BEFORE the result is released. A transition record fits that ordering. A change feed observation cannot, because it happens after the write.

PART 5

Serverless or provisioned

Serverless

- You pay for RU consumed
- No floor, no reservation
- Scales to zero when idle
- Right for a POC, and for spiky volume

Provisioned

- You reserve RU/s and pay for it
- Priced off your PEAK, not your average
- A 400 RU/s minimum you pay regardless
- Right for steady, predictable production load

THE FINDING THAT IS EASY TO MISS At your volume all three partition keys land on the 400 RU/s provisioned minimum. So with a GOOD partition key serverless is roughly four times cheaper than the provisioned floor. With a BAD one you have consumed your way up to the floor and the two are level — the bad design does not look expensive on a provisioned plan, it quietly eats the headroom you needed for growth.

For the POC: serverless. It also gives you a real RU figure to size a provisioned tier with later, which is a better answer than a guess.

SCENARIO

Talk about this

DISCUSS · 7 MIN

You split the case into a hot status document and a cold result document. The hot write succeeds and the cold write fails. What state is the system in?

1. The status says complete and there is no result to read. What does a handler see when they open it?
2. Cosmos transactions work within a single logical partition. Are your two documents in the same one?
3. If they cannot be atomic, what makes the inconsistency detectable — and who is looking?

THE LAB · 45 MINUTES

Three parts, running in parallel

RUN THE EXPERIMENTS

Whoever is comfortable in Python

- python3 run_lab.py — the partition keys
- --throttle — the hot partition
- --split — document shape
- --changefeed — what the audit will miss

MAP THE ACCESS PATTERNS

Anyone not writing Python today

- List every question the system will be asked
- For each: how often, and by whom?
- Rank them by frequency, not by importance
- The top one IS your partition key

DESIGN THE CONTAINER

Pod Lead · Integration · Enterprise

- Partition key, with the reasoning written down
- Document shape: what is hot, what is cold
- Change feed mode, and what it will miss
- Throughput mode, and the review trigger

THE MIDDLE LANE PRODUCES THE ACTUAL DECISION Ranking access patterns by frequency is not a technical task — it needs somebody who knows how the business will use the system. The top of that list is the partition key, and everything else follows.

```
Setup:   unzip   ·   cd into it   ·   python3 run_lab.py
```

THE WORK

What good looks like when you present

Solid — expected at your level

- A container with a chosen partition key
- CRUD working across the lifecycle
- All three query patterns tested
- Change feed enabled
- A monthly cost estimate

Senior — what we are watching for

- The partition key defended against the other two candidates
- You can say what the other patterns cost and how you serve them
- Document shape separates what changes from what does not
- You know what the change feed will NOT capture
- You can explain a 429 on a container with visible headroom
- You can name the access pattern you are least sure about

THE STRONGEST SINGLE ANSWER “We optimised for the pattern that runs 40,000 times a month and accepted the cost on the other two — here is what that cost is.”

SPRINT CONTRIBUTION

The container design that goes into the ADR

Partition key	case_id — with the two rejected candidates named and their cost stated
Document shape	Hot status document, cold result document, and what links them
Query patterns	Which is a point read, which are views, and who maintains the views
Change feed	Enabled, mode chosen, and an explicit note on what it will not capture
Throughput	Serverless for the POC, with the RU figure that will size the next decision
Review trigger	A new access pattern · volume beyond the tested range · partition skew rising

ROW ONE IS THE ONE YOU CANNOT REVISIT Name the rejected candidates. In two years somebody will ask why, and “it seemed right” is not an answer.

REFERENCE

Words used today

Word	Plain meaning	Why it matters here
RU	Request Unit — the unit of work you are billed for	The currency. Latency follows from it, not the reverse
Partition key	The field Cosmos uses to decide where a document lives	Immutable. One access pattern becomes cheap; the rest scan
Logical partition	All documents sharing one partition key value	Cardinality here decides whether you get a hot partition
Physical partition	The machine those documents actually sit on	Capped at roughly 1,000 RU/s regardless of provisioning
Point read	Fetch by id AND partition key	~1 RU. The cheapest operation available
Cross-partition query	A query without the partition key	Costs per document scanned across every partition
Change feed	A stream of documents that changed	Materialised views today, audit trail in Sprint 2
Materialised view	A second copy of the data shaped for a question	How you serve a second access pattern
429	Too many requests — throttled	Often a hot partition, not a capacity shortage

GROUND RULE If a word is used today that is not on this list and it is unclear, ask. An unexplained term is a gap in the explanation, not in anyone's knowledge.

COMMERCIAL CLOSE · 10 MINUTES

“What will the database cost us?”

You can now answer with a number, a design decision behind it, and the figure it would have been if you had chosen differently. Practise all three in four sentences.

01 A partition key makes one pattern a point read and every other one a scan.

02 Cross-partition queries cost per document examined, not per document returned.

03 A hot partition throttles you while the dashboard shows headroom.

04 Separate what changes from what does not. Write RU follows document size.

05 The change feed shows the latest version, not every version. Say so now.

BEFORE YOU LEAVE

Partition key chosen
Document shape split
Change feed enabled
Cost estimated

Sprint 2 turns that change feed into the audit trail you committed to.